OpenClaw: A Comprehensive, Podman-First Guide

Mick Cooney

7 October 2026

Table of Contents

1 Why This Primer Exists **2**
1.1 What This Primer Covers . . . 3
1.2 What This Primer Is Not . . . 3

2 What OpenClaw Is Used For **3**
2.1 What jobs OpenClaw performs in practice . . . 4
2.2 What OpenClaw is not . . . 4

3 How People Actually Use OpenClaw **4**
3.1 Pattern A: Single-user daily assistant . . . 4
3.2 Pattern B: Multi-channel command centre . . . 5
3.3 Pattern C: Local-first with hosted fallback . . . 5
3.4 Pattern D: Specialist agents and nodes . . . 5
3.5 Pattern E: Container-first operations . . . 5

4 Ideas for How You Could Use OpenClaw **6**

5 OpenClaw Architecture in One Mental Model **6**
5.1 The message path . . . 6
5.2 State locations that matter . . . 7

6 Comprehensive Local Setup (Podman-First) **7**
6.1 Deployment goals . . . 7
6.2 Prerequisites . . . 7
6.3 Install and verify the host CLI . . . 8
6.4 Bootstrap the Podman Gateway . . . 8
6.5 Manage the container from the host CLI . . . 8
6.6 Persistence and ports . . . 9
6.7 Optional Quadlet mode . . . 9
6.8 Upgrades and recovery . . . 9
6.9 Day-two operations . . . 9

7 Channels, Sessions, and Access Control **10**

8 Models, Providers, and Failover **10**
8.1 Ollama: use the native API . . . 10
8.2 Generic OpenAI-compatible providers . . . 11
8.3 Choosing a local model . . . 11
8.4 Constrained hardware . . . 12

9 OpenClaw and Hermes Agent Server **12**
9.1 When to use OpenClaw . . . 13
9.2 When to use Hermes Server . . . 13
9.3 Combining them . . . 14

10 Tool Sandboxing and Containment **14**

11 Operational Reference **15**
11.1 Security checklist . . . 15
11.2 Troubleshooting order . . . 15
11.3 Exposure profiles . . . 15

12 Runbooks **16**
12.1 Deterministic bring-up . . . 16
12.2 Backup . . . 16
12.3 Validate after restore . . . 17

13 Reference Links **17**

14 Closing Perspective **17**

Long-form edition · 7 October 2026. Runtime requirements rechecked against official documentation.

---

1 Why This Primer Exists

OpenClaw is easy to describe badly. Calling it a chatbot hides the important part; calling it an autonomous agent makes it sound like a single model. OpenClaw is a self-hosted Gateway: one long-lived service that connects chat channels, control-plane clients, agent sessions, model providers, tools, and device nodes.

The first successful conversation is not the difficult milestone. The difficult work starts when you need to decide which channels may reach the agent, where tool calls execute, how model failures are handled, what state survives a restart, and how to recover without guessing. Short installation guides rarely explain those boundaries.

This primer gives you an operational mental model and a Podman-first deployment path. It also separates OpenClaw from Hermes Agent Server, a neighboring project with different server processes and state semantics. The comparison matters because both projects are now used as self-hosted assistants, but they are not interchangeable components.

1.1 What This Primer Covers

You will learn:

1. What the OpenClaw Gateway owns and how messages move through it.
2. How to install and operate OpenClaw on a host or in a rootless Podman container.
3. How channels, sessions, model selection, fallbacks, memory, and tool sandboxes fit together.
4. How to run local models through Ollama or another OpenAI-compatible service.
5. How OpenClaw compares with Hermes Agent Server and when the two can be combined.

1.2 What This Primer Is Not

This is not a catalogue of every channel plugin, a model benchmark, or a security certification. Provider names, model names, CLI flags, and plugin behaviour change quickly; use the linked official documentation to verify a command before applying it to a production host. The examples are patterns you can adapt, not a claim that one configuration is safe for every environment.

2 What OpenClaw Is Used For

OpenClaw is a self-hosted assistant Gateway for developers and power users who want one agent reachable through several surfaces. The current project describes a single Gateway serving messaging surfaces such as Discord, Google Chat, iMessage, Matrix, Microsoft Teams, Signal, Slack, Telegram, WhatsApp, and WebChat, along with control UI, CLI, and mobile or desktop nodes. Some surfaces ship in core; others are installed as official channel plugins.

The Gateway is the source of truth for sessions, routing, and channel connections. It maintains provider connections, exposes a typed WebSocket control protocol, validates inbound frames, emits lifecycle events, and runs the agent loop that turns model output into tool calls and replies.

That makes OpenClaw a control plane for agent work rather than a model server. It can talk to hosted providers, local model servers, or both. It can route different agents or sessions to

different models. It can put tool execution in a separate sandbox. It does not itself provide the model weights or guarantee that a model's output is safe.

2.1 What jobs OpenClaw performs in practice

OpenClaw handles channel ingress and egress, identity and pairing, session construction, media handling, model selection, authentication-profile rotation, model fallback, tool policy, sandbox dispatch, workspace access, cron jobs, webhooks, and operational diagnostics.

The distinction between those jobs is useful when troubleshooting. A Telegram pairing failure is a channel problem. A provider authentication failure is a model-provider problem. A command that ran in the wrong filesystem is an execution-policy problem. A session that disappeared after a rebuild is a state-persistence problem.

2.2 What OpenClaw is not

OpenClaw is not:

- a large language model or model-weight distribution;
- a replacement for Ollama, vLLM, LM Studio, or another inference server;
- a security boundary merely because the Gateway is running in a container;
- a magic memory system that makes every conversation permanently available;
- the same project as Hermes Agent, Hermes Server, or the Hermes model family.

Containerizing the Gateway can protect the Gateway process and its files, but OpenClaw's own tool sandbox is a separate setting. If you enable no sandbox, tool execution may still occur on the host or in the Gateway container, depending on your configuration.

3 How People Actually Use OpenClaw

3.1 Pattern A: Single-user daily assistant

Start with one Gateway, one agent, one model provider, and the Control UI. This gives you a small system whose failures are easy to inspect. Use it for drafting, summarisation, research notes, coding assistance, and low-risk local automation.

The useful discipline is to establish a working session before adding channels or specialist agents. If the basic chat cannot complete a normal request, adding messaging integrations only multiplies the places where the failure can appear.

3.2 Pattern B: Multi-channel command centre

Once the local path is stable, connect the channels you actually use. One Gateway can serve multiple channel plugins, but each channel introduces a new trust boundary: sender identity, pairing, group mentions, attachments, rate limits, and outbound delivery.

Keep the initial policy narrow. Allow only known senders, require mentions in group chats, and keep the Gateway's published ports on loopback unless you have an explicit remote-access design such as a tailnet or authenticated proxy.

3.3 Pattern C: Local-first with hosted fallback

A local model can be the default for privacy and cost while a hosted provider remains available for difficult requests or local-service outages. OpenClaw's fallback behaviour has two stages: it rotates usable authentication profiles within the current provider, then tries the configured model fallbacks.

Fallbacks are not always applied to explicit user selections. A configured default can use `agents.defaults.model.fallbacks`; an explicit session model is strict unless that selection has its own fallback policy. When OpenClaw automatically moves to a fallback, that automatic state can persist across subsequent turns while the original primary is periodically reprobed; it is cleared when the primary recovers. Treat this as a reliability feature, not as permission to hide provider errors.

3.4 Pattern D: Specialist agents and nodes

Separate agents can have distinct workspaces, models, and routing bindings. Nodes can expose device capabilities such as camera, screen, voice, or location to the Gateway. This is powerful because the assistant can move from text to action, but it also increases the number of capabilities that need pairing and review.

3.5 Pattern E: Container-first operations

Rootless Podman is a good fit when you want a narrow host contract, explicit bind mounts, and user-level service management. The official Podman path runs the Gateway in a container while the host `openclaw` CLI remains the management surface. It is different from running every tool call in a sandbox; you can choose that separately.

4 Ideas for How You Could Use OpenClaw

For documentation work, OpenClaw can summarise a repository, draft release notes, and enforce a project style guide while leaving final changes under review. For engineering triage, it can classify incoming issues and prepare a response without granting permission to merge or deploy.

For personal research, it can act as a persistent synthesis layer across channels and sessions. For homelab operations, it can turn health checks and logs into a digestible daily report. For mobile workflows, a paired node can supply a camera capture or voice interaction while the Gateway retains the session and policy.

The safe design pattern is consistent: let the model propose work, let OpenClaw validate the request and route it, and let an explicit execution boundary decide what the tool can actually touch.

5 OpenClaw Architecture in One Mental Model

Think in five layers:

1. **Channel and node adapters** receive messages, media, or device events.
2. **The Gateway** owns routing, sessions, provider connections, WebSocket RPC, and lifecycle events.
3. **The agent runtime** builds context, calls the selected model, interprets tool requests, and produces replies.
4. **Providers and model services** generate model output through hosted APIs or local endpoints.
5. **Tool execution and state** perform actions and persist configuration, credentials, sessions, workspaces, memory, and logs.

The Gateway is a long-lived process. Control-plane clients such as the CLI, web UI, and desktop app connect to it over WebSocket, normally on `127.0.0.1:18789`. Nodes connect over the same general transport with an explicit node role and declared capabilities. A single Gateway should own a host's messaging sessions; starting multiple competing Gateways against the same state creates confusing locks and duplicate channel connections.

5.1 The message path

A useful trace for a request is:

```
channel or UI
  -> Gateway ingress and authorization
```

```
-> session resolution and routing
-> provider/model request
-> validated tool call, if any
-> sandbox or configured execution target
-> tool result added to context
-> model response
-> Gateway delivery to the originating surface
```

The model is only one stage in this path. When a task fails, identify the last successful stage before changing prompts or models.

5.2 State locations that matter

The default state directory is `~/.openclaw`. It contains the main configuration, agent-specific state, auth routing and credential material, channel state, sessions, SQLite databases, and the default workspace under `~/.openclaw/workspace`. Those databases are authoritative state, not disposable caches. A container image is disposable; the mounted state directory is the durable system.

Do not treat the workspace as a backup of the entire installation. Back up configuration, auth material, session data, memory, and workspace together, and protect the archive as you would protect API credentials.

6 Comprehensive Local Setup (Podman-First)

This section follows the current official rootless Podman workflow. The commands below assume a Linux host with Podman and an OpenClaw checkout. For a quick non-container install, use the official installer and `openclaw onboard` instead.

6.1 Deployment goals

The target posture is a rootless Gateway container, host-controlled state, loopback-only published ports, user-level restart management when needed, and a separate tool sandbox policy for non-main sessions.

6.2 Prerequisites

You need rootless Podman and the OpenClaw CLI on the host. As checked on 7 October 2026, the Node guide requires Node 24.16+ or 26.1+ with a WAL-reset-safe linked SQLite library; Node 26 is recommended. Node 22, 23, and 25 are unsupported. The compatibility guide explains the SQLite safety checks: a version number alone is insufficient for a distro build linked to a different SQLite library. Upgrade the runtime before updating OpenClaw.

The installer can provision a compatible runtime. `systemd --user` is optional for Quadlet-managed startup.

6.3 Install and verify the host CLI

The installer is the shortest path for a normal host install:

```
curl -fsSL https://openclaw.ai/install.sh | bash
openclaw --version
openclaw doctor
```

Also check `node --version` for a manually managed install. Keep the host CLI's runtime and the Gateway image's runtime compatible with the selected release; upgrading host Node does not upgrade a running container. The Podman guide documents rebuilding or pulling the image and restarting the Gateway.

If you want a checkout-based installation, follow the repository's current install instructions. Do not assume that an old `pnpm` or Node version from an earlier guide remains supported.

6.4 Bootstrap the Podman Gateway

From the OpenClaw repository root:

```
./scripts/podman/setup.sh
./scripts/run-openclaw-podman.sh launch
./scripts/run-openclaw-podman.sh launch setup
```

The setup helper builds or selects the Gateway image, creates `~/.openclaw/openclaw.json` if needed, and creates `~/.openclaw/.env` with a Gateway token if one is not present. The launcher uses the current user namespace and mounts OpenClaw state into the container.

Open the Control UI at `http://127.0.0.1:18789/`. For a container install, complete provider authentication through OpenClaw's own setup so credentials are stored in the mounted OpenClaw state. Do not assume that a host login in `~/.claude` or `~/.codex` is visible inside the container.

6.5 Manage the container from the host CLI

Use the container-aware CLI explicitly while you learn the setup:

```
openclaw --container openclaw gateway status --deep
openclaw --container openclaw doctor
openclaw --container openclaw dashboard --no-open
openclaw --container openclaw channels login
```

If you operate one named container repeatedly, `OPENCLAW_CONTAINER=openclaw` can be used as the documented shortcut. Verify the target before making configuration changes; a healthy CLI command against the wrong Gateway is still the wrong result.

6.6 Persistence and ports

The official launcher persists the configuration directory and workspace from the host. The normal published Gateway port is `18789`; the bridge port is `18790`. Keep both published on `127.0.0.1` unless your remote-access plan explicitly protects them.

The token in `~/.openclaw/.env` is a secret. Do not put it in a checked-in compose file, shell history, screenshot, or support ticket. If you change the host config or workspace paths, pass the same values to both setup and launch; otherwise the two commands can operate on different state.

6.7 Optional Quadlet mode

On a Linux host with `systemd --user`, run setup with Quadlet enabled:

```
./scripts/podman/setup.sh --quadlet
systemctl --user start openclaw.service
systemctl --user status openclaw.service
journalctl --user -u openclaw.service -f
```

For a headless host that must start the service after reboot, user lingering may be required:

```
sudo loginctl enable-linger "$(whoami)"
```

After editing the generated unit, reload and restart it:

```
systemctl --user daemon-reload
systemctl --user restart openclaw.service
```

6.8 Upgrades and recovery

When you rebuild or pull a newer image, restart the container or Quadlet service. If the updated Gateway exits while applying state changes, run a one-off `podman run` with the same image, user mapping, and mounted state, ending with `openclaw doctor --fix`, then start the Gateway normally. The official Podman guide contains the complete command because the exact image and mount paths must match your deployment. Afterward, run a linting preflight:

```
openclaw --container openclaw doctor --lint --json
openclaw --container openclaw gateway status --deep
```

Do not "fix" an upgrade by deleting `~/.openclaw`. That directory is the system's durable state, not a cache.

6.9 Day-two operations

```
podman logs -f openclaw
podman ps --filter name=openclaw
podman stop openclaw
./scripts/run-openclaw-podman.sh launch
openclaw --container openclaw gateway status --deep
```

7 Channels, Sessions, and Access Control

Channels are not just output formats. They are authenticated ingress paths into an agent with tools. Start with one channel and configure the narrowest useful policy: known-user allowlists, pairing where supported, mention requirements in groups, and no public port exposure.

The Control UI is useful for local administration because it gives you a direct view of sessions and configuration. Remote access should use an explicit access layer such as Tailscale or an authenticated proxy. A port that is reachable from a LAN is not "local" merely because the agent is personal.

Sessions are the unit of conversational state. Main, group, channel, and agent-specific sessions can have different routing and sandbox behaviour. Do not infer that two messages share context because they reached the same Gateway; inspect the session key and routing policy.

8 Models, Providers, and Failover

A model reference has the form `provider/model`. It chooses a provider and model; it does not by itself decide the low-level agent runtime. The primary model normally comes from `agents.defaults.model.primary`, and configured fallbacks are tried in order. Authentication-profile rotation happens inside a provider before OpenClaw advances to the next model.

Fallback execution is initiated for the current turn, but an automatically selected fallback can remain the active automatic session state across later turns while the original primary is reprobed. Explicit user model choices are intentionally strict, so a visible provider error can be preferable to an unexpected answer from a different model.

8.1 Ollama: use the native API

OpenClaw's Ollama provider uses Ollama's native API, not the OpenAI-compatible `/v1` endpoint. In a Podman Gateway container, `127.0.0.1` means the Gateway container itself, not the host running Ollama. Point the provider at a host or LAN address reachable from the container. For runtimes that provide it, `host.containers.internal` is a useful starting point:

```
{
  models: {
    providers: {
      ollama: {
        baseUrl: "http://host.containers.internal:11434",
        api: "ollama",
```

```
        apiKey: "ollama-local"
      }
    }
  },
  agents: {
    defaults: {
      model: {
        primary: "ollama/gemma4",
        fallbacks: ["ollama/qwen3.5"]
      }
    }
  }
}
```

Do not append `/v1` to that URL. OpenClaw documents that the compatibility endpoint can break tool calling and cause raw tool-call JSON to appear as plain text. Ensure Ollama listens on an interface the container can reach without exposing it more broadly than intended; test connectivity from the Gateway container. A non-loopback custom URL may also require explicit model configuration because automatic discovery is not guaranteed. Local or private Ollama hosts use the `ollama-local` marker; public Ollama Cloud endpoints require a real credential and should use the dedicated cloud provider path where appropriate.

Onboarding can discover installed models and check tool support and context metadata. It does not automatically solve a model that is too small, lacks reliable tool calling, or cannot hold the working context of your task.

8.2 Generic OpenAI-compatible providers

LM Studio, vLLM, LiteLLM, and many hosted gateways expose OpenAI-compatible endpoints. The provider configuration is not interchangeable with Ollama's native configuration: use the endpoint and API mode documented by the service, then verify one real tool call before trusting the integration.

8.3 Choosing a local model

For an agentic workload, evaluate more than tokens per second:

- tool-call schema reliability;
- context window under accumulated tool results;
- first-token latency and long-context prefill;
- memory and VRAM pressure during key-value-cache growth;
- recovery behaviour when the model service restarts.

Do not promise a fixed VRAM number for a model family. Quantisation, context length, vision support, batching, and backend version all change the footprint. Measure the exact

model and settings on the hardware you intend to operate.

8.4 Constrained hardware

Partial GPU offload can make a local model usable on a laptop that cannot hold the full model in VRAM. The tradeoff is lower throughput and a more complicated memory budget. The key-value cache can still consume significant VRAM as context grows even when model weights are split between GPU and CPU RAM.

Start with a modest context window, measure memory during a long tool loop, and increase it only when the workload requires it. A nominal "64K context" setting is not free: it changes both latency and memory behaviour.

9 OpenClaw and Hermes Agent Server

OpenClaw and Hermes are frequently compared because both can power a self-hosted assistant. The precise comparison is OpenClaw Gateway versus the server surfaces of Hermes Agent. Hermes is not merely a model backend, and "Hermes Server" is not one single process in the official terminology.

Hermes has at least three relevant long-running modes:

1. `hermes gateway` connects messaging platforms, runs sessions, and handles scheduled jobs.
2. The API server, enabled through `API_SERVER_ENABLED=true` and protected with `API_SERVER_KEY`, exposes an OpenAI-compatible HTTP API on port `8642` by default.
3. `hermes serve` is a headless backend for the desktop application's JSON-RPC/WebSocket interface, commonly listening on port `9119`.

Dimension	OpenClaw Gateway	Hermes Agent server surfaces
Primary role	Multi-channel Gateway and agent control plane	Agent runtime exposed through messaging, HTTP API, or desktop backend
Main control transport	Typed WebSocket API, Control UI, CLI, nodes	OpenAI-compatible HTTP API; `hermes serve` JSON-RPC/WebSocket; messaging adapters

Dimension	OpenClaw Gateway	Hermes Agent server surfaces
Default local web surface	Control UI on `127.0.0.1:18789`	API server on `127.0.0.1:8642`; desktop backend commonly on `9119`
Durable state	`~/.openclaw` configuration, SQLite databases, sessions, auth, workspaces, and agent state	`~/.hermes` config, secrets, profiles, memories, skills, cron, sessions, logs, and `state.db`
Execution boundary	Sandbox is off by default; Docker, Podman, SSH, or OpenShell can be selected for tools	Local, Docker, SSH, Singularity, Modal, Daytona, Vercel Sandbox, or other configured backends
Messaging	Channel plugins owned by the Gateway	A separate `hermes gateway` process connects many messaging platforms
Model routing	Provider/model refs, auth-profile rotation, configured fallbacks	Provider configuration and model profiles; routing is configured in Hermes

Neither product becomes safe merely because its HTTP port is bound to localhost. Both can run commands, read files, use credentials, and access external services. Choose the system whose session, plugin, memory, and execution model matches your workflow, then secure its actual trust boundaries.

9.1 When to use OpenClaw

Choose OpenClaw when one Gateway serving many channels, device nodes, WebChat, and a typed control-plane protocol are central requirements. Its Podman workflow is also attractive when you want the Gateway container and host CLI to have clearly defined roles.

9.2 When to use Hermes Server

Choose Hermes when you want the Hermes Agent runtime's built-in memory, skills, toolsets, profiles, and messaging gateway, or when you want to attach Open WebUI or another OpenAI-compatible frontend directly to a tool-using Hermes backend.

9.3 Combining them

An OpenAI-compatible Hermes API can look like a model endpoint to another system, but that does not mean the two systems share sessions or tool policy. This is an experimental generic OpenAI-compatible integration pattern, not a documented supported OpenClaw-plus-Hermes pairing. Hermes returns a tool-using agent's final response; it is not a plain inference server. If OpenClaw sends a request to Hermes, decide explicitly which system owns tool execution, memory, authorization, and the user-visible session. Avoid building two independent agents that both believe they own the same terminal or credentials.

10 Tool Sandboxing and Containment

OpenClaw separates Gateway placement from tool placement. The Gateway can run in a Podman container while tool execution uses a distinct Podman sandbox, or the Gateway can run on the host while tools use a container. Sandboxing is off by default; the relevant settings are `mode`, `scope`, and `backend`.

The useful starting point for a multi-channel assistant is to sandbox all sessions unless you have deliberately trusted the main-session routing:

```
{
  agents: {
    defaults: {
      sandbox: {
        mode: "all",
        scope: "session",
        backend: "podman"
      }
    }
  }
}
```

`non-main` protects group and channel sessions only when those messages do not converge on the main session; direct messages commonly do converge there. Use `all` when channel input is not fully trusted. `scope: "session"` gives each sandboxed session its own runtime; `scope: "agent"` shares one sandbox across that agent's sandboxed sessions. A Podman sandbox also requires the sandbox image, a compatible host-Podman connection, and consistent host-path mounts; browser sandboxing remains Docker-only. Remember that elevated tools can bypass the sandbox, and that the Gateway itself remains outside the tool sandbox.

For higher assurance, remove unnecessary workspace mounts, keep network access constrained, use deny-by-default tool policy for exposed channels, and test the exact operations the assistant is allowed to perform. A sandbox reduces blast radius; it does not turn unreviewed prompts into a formal security proof.

11 Operational Reference

11.1 Security checklist

1. Keep Gateway and bridge ports on loopback by default.
2. Use pairing or explicit allowlists for every messaging surface.
3. Require mentions in group chats unless the group is fully trusted.
4. Keep tokens in the OpenClaw state environment file or a supported secret store, never in committed configuration.
5. Enable non-main or all-session sandboxing before exposing channels.
6. Review workspace mounts and elevated-tool permissions as a single policy.
7. Run `openclaw doctor` after upgrades or configuration migrations.
8. Back up `~/.openclaw` before changing provider, channel, or agent state.
9. Test recovery from a provider outage and a container replacement.

11.2 Troubleshooting order

Start at the outside and move inward:

1. **Reachability:** Is the Gateway process running, and is the expected port bound?
2. **Authorization:** Does the caller have a valid token, pairing, or allowlist entry?
3. **Session:** Did the message route to the expected agent and session key?
4. **Provider:** Can the selected model authenticate and answer a minimal request?
5. **Tool policy:** Is the requested tool enabled and permitted in this session?
6. **Execution:** Did the sandbox or host backend run the command in the expected directory?
7. **State:** Did the result persist to the mounted configuration, workspace, and session directories?

When tool calls arrive as plain text, inspect the provider API mode first. When a container restart loses configuration, inspect mounts and paths before re-running onboarding.

11.3 Exposure profiles

This matrix compares the controls to apply at each exposure level, from local development to public reachability.

Control	Local development	Trusted home or tailnet	Publicly reachable
Published ports	Loopback	Loopback plus controlled tailnet access	Authenticated proxy or tailnet; no raw Gateway port
Channel access	One known user	Pairing plus allowlists	Strict allowlists and monitoring
Sandbox	Non-main while testing	Non-main or all	All sessions, narrow tool policy
Workspace	Deliberate read/write	Prefer narrow mounts	None by default
Model fallback	Simple	Explicit chain	Explicit chain plus outage monitoring
Backups	Manual	Scheduled local	Scheduled encrypted off-host copies

12 Runbooks

12.1 Deterministic bring-up

```
./scripts/podman/setup.sh
./scripts/run-openclaw-podman.sh launch
./scripts/run-openclaw-podman.sh launch setup
openclaw --container openclaw gateway status --deep
openclaw --container openclaw doctor
openclaw --container openclaw models list
```

Connect one channel, send one normal request, perform one deliberately harmless tool call, and confirm the result appears in the expected workspace. Only then add additional channels, nodes, or providers.

12.2 Backup

Use OpenClaw's backup command for live state. It understands the authoritative SQLite databases and verifies the resulting archive:

```
openclaw backup create \
  --output "$HOME/openclaw-state-$(date +%Y%m%d_%H%M%S).tar.gz" \
  --verify
```

Store the archive somewhere access-controlled. It can contain tokens, auth state, private conversations, and tool results. Do not copy live SQLite files with a raw `tar` command while the Gateway is writing to them.

12.3 Validate after restore

```
openclaw --container openclaw doctor --lint --json
openclaw --container openclaw gateway status --deep
openclaw --container openclaw models list
```

Send a test message through the Control UI before reconnecting every external channel. This isolates a state or provider problem from a channel-authentication problem.

13 Reference Links

- OpenClaw documentation
- OpenClaw installation
- OpenClaw Podman deployment
- Gateway architecture
- Models and model selection
- Model failover
- Ollama provider
- Sandboxing
- OpenClaw repository
- Hermes Agent documentation
- Hermes API server
- Hermes desktop and `hermes serve`

14 Closing Perspective

OpenClaw becomes dependable when you stop treating it as a chat window and start treating it as a small distributed system. The Gateway owns connectivity and state; the model proposes reasoning and actions; the tool boundary decides what can happen; the operator owns secrets, access, and recovery.

Rootless Podman is useful because it makes those boundaries visible. It does not remove the need for channel authorization, sandbox policy, backups, or upgrade testing. If you can name the process, state directory, port, credential, and execution target for every part of your setup, you are operating OpenClaw rather than merely hoping it works.